EXPERIMENT 1 : Inter Process Communication

Date Of Performance:

Date Of Submission:

1. Aim

To study and implement **Inter Process Communication (IPC)** mechanisms to enable data exchange and coordination between multiple processes using pipes in a distributed computing environment.

OBJECTIVES

1. To understand the need and importance of Interprocess Communication in operating systems and distributed environments.
2. To compare different IPC mechanisms based on performance, scalability, reliability, and ease of implementation.
3. To gain conceptual knowledge required for implementing IPC techniques in real-world system applications.
4. To analyze how processes communicate and synchronize with each other.
5. To implement a basic IPC mechanism for sending and receiving data between processes.
6. To observe the role of IPC in improving process coordination, efficiency, and reliability in distributed systems.

SCOPE OF THE STUDY

Scope1: Analysis and Evaluation of Interprocess Communication (IPC) Mechanisms in Operating Systems and Distributed Systems

The first scope of this study focuses on exploring the role and necessity of Interprocess Communication (IPC) in both operating systems and distributed computing environments. It covers how processes exchange data and synchronize their activities to ensure efficient system performance and coordination. The scope includes examining various IPC mechanisms such as pipes, message queues, shared memory, and sockets, highlighting their working principles and practical applications. Furthermore, it involves a comparative analysis of these mechanisms based on key factors like performance, scalability, reliability, and ease of implementation. The study also considers how different environments—single-system OS versus distributed systems—affect the choice and efficiency of IPC methods, aiming to provide insights into selecting the most suitable communication technique for specific use cases.

Scope2: Conceptual Understanding and Practical Analysis of Process Communication and Synchronization in IPC

The second scope of this emphasizes building a strong conceptual foundation for implementing Interprocess Communication (IPC) techniques in real-world system applications. It focuses on understanding how processes interact, exchange information, and coordinate their execution through various communication methods. The study examines the underlying concepts required to design and implement IPC mechanisms effectively, including data sharing, message passing, and synchronization techniques such as semaphores and signals. Additionally, it analyzes how processes maintain consistency and avoid conflicts while communicating concurrently. By bridging theoretical knowledge with practical application, this scope aims to prepare learners to develop efficient, reliable, and well-coordinated process communication systems in real-world computing environments.

Scope3: Implementation and Evaluation of Basic IPC Mechanisms for Efficient Process Coordination

The third scope of this focuses on the practical implementation of a basic Interprocess Communication (IPC) mechanism to enable data exchange between processes. It involves designing and developing simple communication models where processes can send and receive information effectively using suitable IPC techniques. Additionally, the scope includes observing and analyzing how IPC contributes to improved process coordination, system efficiency, and reliability, especially in distributed environments. By evaluating real-time communication scenarios, this study highlights the importance of IPC in ensuring smooth interaction between processes, reducing conflicts, and enhancing overall system performance.

2. THEORY

2.1 Inter process Communication in Distributed Computing:

Distributed computing is a type of computing approach where multiple independent computers, known as nodes, are linked together through a network to work together on a shared objective. Each of these nodes has its own processing unit and memory space, and they communicate with each other by sending and receiving messages.

In such systems, there isn't a single, shared memory that all the nodes can access directly. Because of this, communication between the nodes becomes essential. Inter Process Communication, or IPC, is the set of methods and tools that allow different processes to share information, ensure their actions happen in the right order, and work together smoothly to accomplish their tasks.

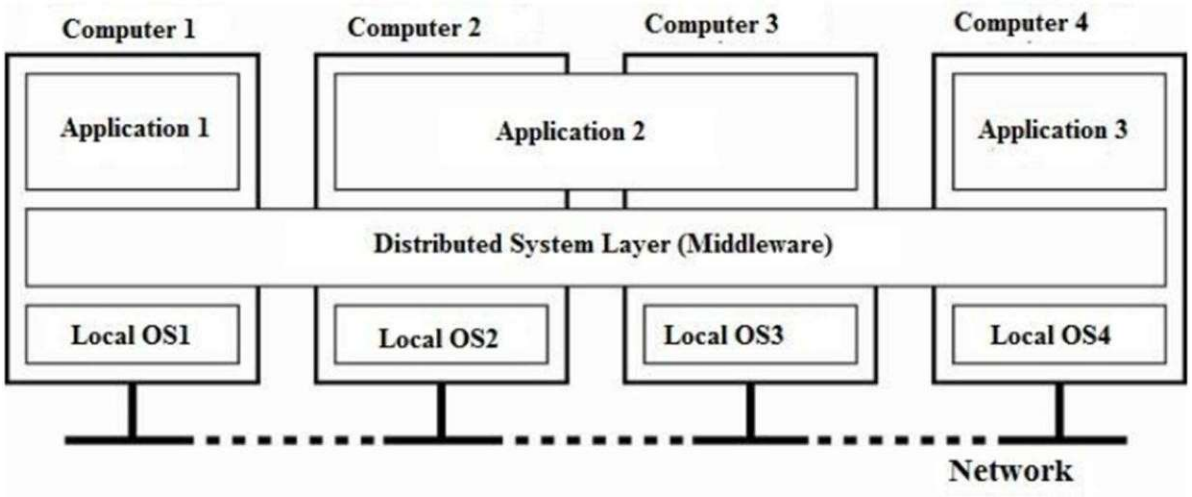


Fig.1.1 Distributed System Architecture

Fig 1.1 shows a **distributed system architecture** ,There are multiple computers (Computer 1 to 4), each having its own **local operating system (OS)**.Different **applications** run on different computers.A **Distributed System Layer (Middleware)** sits between applications and OS.

Inter Process Communication (IPC) refers to a set of techniques and mechanisms that allow multiple processes to communicate with each other and share data. IPC is required when processes are running concurrently and need to exchange data or signals to complete a distributed task.

IPC can be implemented between:

- Processes on the same system
- Processes on different systems connected via a network

In distributed computing, processes are independent and may execute on different machines. IPC ensures:

1. Data sharing between distributed processes
2. Coordination and synchronization of concurrent processes
3. Consistency of data across distributed nodes
4. Efficient utilization of resources
5. Fault tolerance and reliability

2.2 IPC Mechanisms :

- **Pipes**

  Pipes provide a simple mechanism for communication between related processes. They support unidirectional data flow in a FIFO manner. Anonymous pipes are used between parent and child processes. Named pipes allow communication between unrelated processes. The operating system manages the pipe buffer. Pipes are suitable only for local inter-process communication.

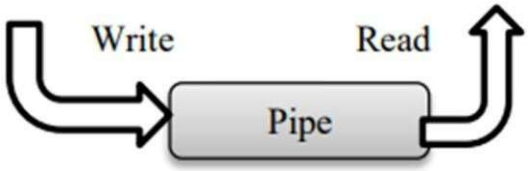


Fig.2.2.1 Ordinary Pipe

Figure 2.2.1 represents a **Pipe mechanism (IPC – Inter Process Communication)**,A **pipe** acts like a communication channel between processes,One process **writes data** into the pipe.Another process **reads data** from the pipe.

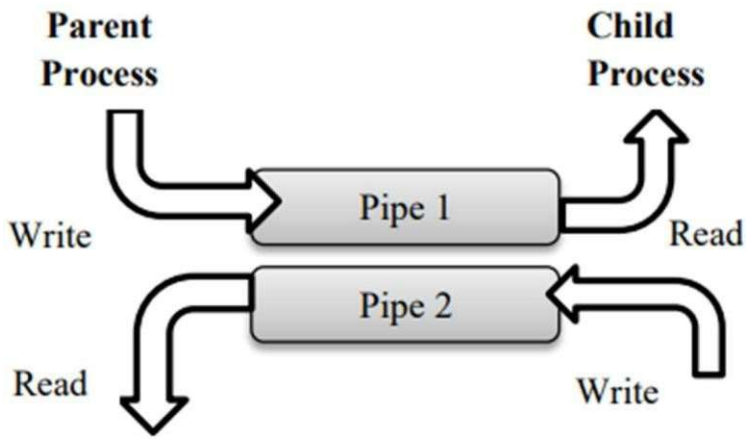


Fig.2.2.2 Named Pipes

Figure2.2.2 shows **two-way communication using pipes between a parent and child process**

Pipe 1
Parent process **writes** data
Child process **reads** data
Pipe 2
Child process **writes** data
Parent process **reads** data

- **Message Queues**

  Message queues allow processes to communicate by sending messages to a queue. Messages remain in the queue until read by the receiver. Communication is asynchronous in nature. Messages can have priorities for ordered processing. The sender and receiver need not run simultaneously. Message queues are widely used in distributed applications.

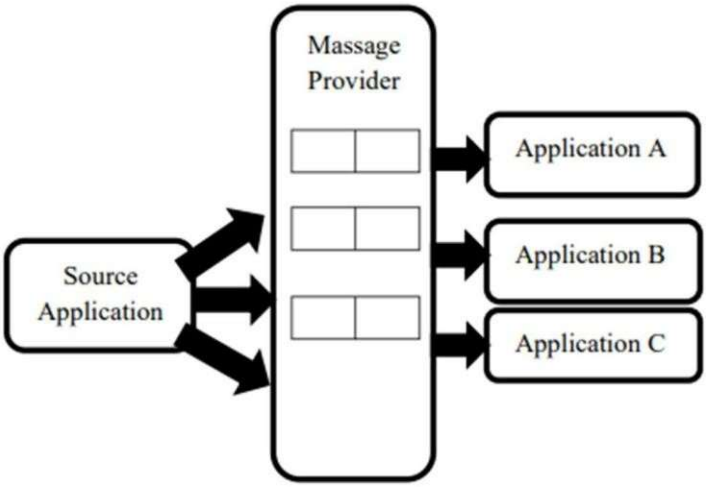


Fig.2.2.3 Point to point message queue based

Figure 2.2.3 represents a **Message Queue (Message Passing System)**.A **Source Application** sends messages to a **Message Provider (queue/broker)**.The message provider stores messages temporarily. Multiple applications (**A, B, C**) can **receive messages** from the queue.

- **Shared Memory**

  Shared memory enables multiple processes to access a common memory space. It provides the fastest IPC mechanism since no kernel involvement is needed after setup. Processes directly read and write data in shared memory. Synchronization is required to avoid race conditions. Semaphores or mutexes are commonly used for control. Shared memory is mainly applicable to processes on the same system.

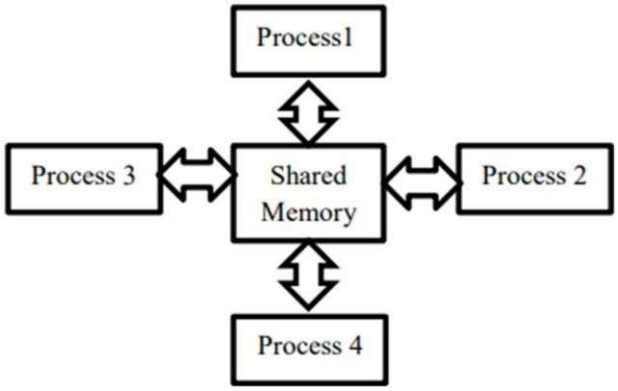


Fig.2.2.4 Shared Memory

Figure 2.2.4 shows **Shared Memory IPC (Inter-Process Communication)**.Multiple processes (Process 1, 2, 3, 4) are connected to a **common shared memory space**.All processes can **read and write data directly** in this shared area.

- **Semaphores**

Semaphores are synchronization tools used in IPC. They control access to shared resources. Semaphores prevent race conditions and deadlocks. Binary and counting semaphores are commonly used types. They use wait and signal operations. Semaphores ensure safe and orderly process execution.

- **Sockets**

Sockets enable communication between processes over a network. They support communication between systems in distributed environments. Sockets follow the client– server model. TCP sockets provide reliable communication. UDP sockets provide faster but unreliable communication. Sockets are widely used in web and cloud applications.

- **RPC**

RPC allows a process to invoke a procedure on a remote system. The call appears as a local function call to the programmer. RPC hides the complexity of network communication. Client and server communicate using stubs. It simplifies distributed application development. RPC is widely used in microservices and distributed systems.

2.3 Algorithm

Server side algorithm
INPUT : Request from client
OUTPUT : Response sent to client

BEGIN:

1. Initialize server
2. Bind server to port and start listening
3. While (server is running) do
4. Accept client connection
5. Receive request Req from client
6. Process Req
7. Generate response Res
8. Send Res to client
9. Close client connection (optional)
10. End While

END

Client Side Algorithm:

INPUT : User request / input
OUTPUT : Response received from server

BEGIN:

1. Initialize client application
2. Accept input from user
3. Create request message Req
4. Establish connection with server
5. Send Req to server
6. Wait for response from server
7. Receive response Res
8. Process and display Res to user
9. Terminate connection

END

Functions used:

1. socket.socket()

This function creates a new socket for communication between processes. AF_INET specifies IPv4 addressing, and SOCK_STREAM specifies TCP protocol. It enables reliable, connection-oriented communication.

2. bind()

The bind() function assigns an IP address and port number to the server socket. It ensures that the server listens on a known address. Binding is mandatory for server-side communication.

3. listen()

The listen() function puts the server socket into passive mode. It allows the server to wait for incoming client connections. The operating system manages a queue of connection requests. It is used only for TCP communication.

4. accept()

The accept() function accepts an incoming client connection. It returns a new socket object and the client's address. Communication with the client occurs through the new socket.

5. connect()

The connect() function is used by the client to establish a connection with the server. It requires the server's IP address and port number. Once connected, data exchange can begin.

6. send()

The send() function transmits data from one process to another. Data is sent in byte format over the socket. It is used by both client and server. Reliable delivery is ensured in TCP sockets.

7. recv()

The recv() function receives data sent by the other process. It specifies the maximum number of bytes to receive. The received data is in byte format.

8. close()

The close() function terminates the socket connection. It releases system resources associated with the socket. It is used by both client and server. Proper closure prevents resource leakage.

3. IMPLEMENTATION

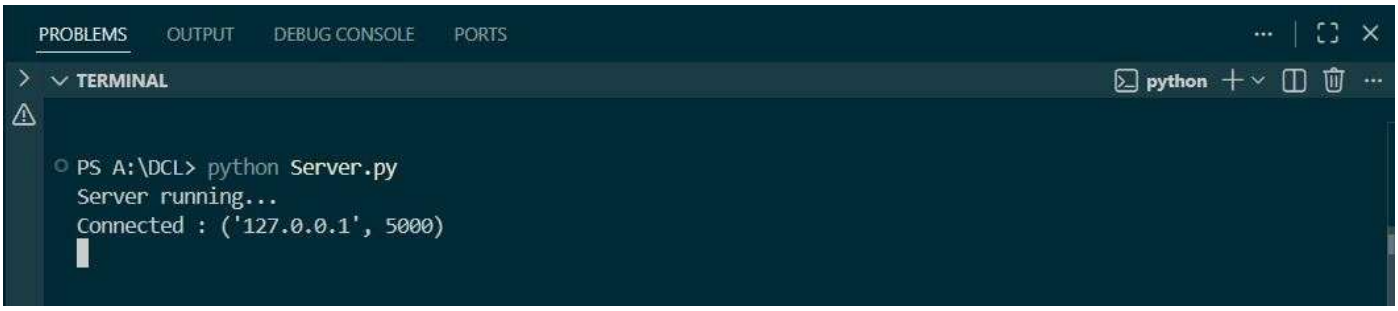
Programming Language Used: Python.
Server Used: https://www.programiz.com/python-programming/online-compiler/

Server code

```
import socket
import os

HOST = '127.0.0.1'
PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen()

print("Server running...")

while True:
    conn, addr = server.accept()
    print(f"Connected:  {addr}")

    file_name = conn.recv(1024).decode()

    if os.path.exists(file_name):
        with open(file_name, 'r') as f:
            content = f.read()
        conn.send(f"FILE_FOUND\n{content}".encode())
    else:
        conn.send(f"FILE_NOT_FOUND: {file_name}".encode())

    conn.close()
```

OUTPUT

Command: py server.py

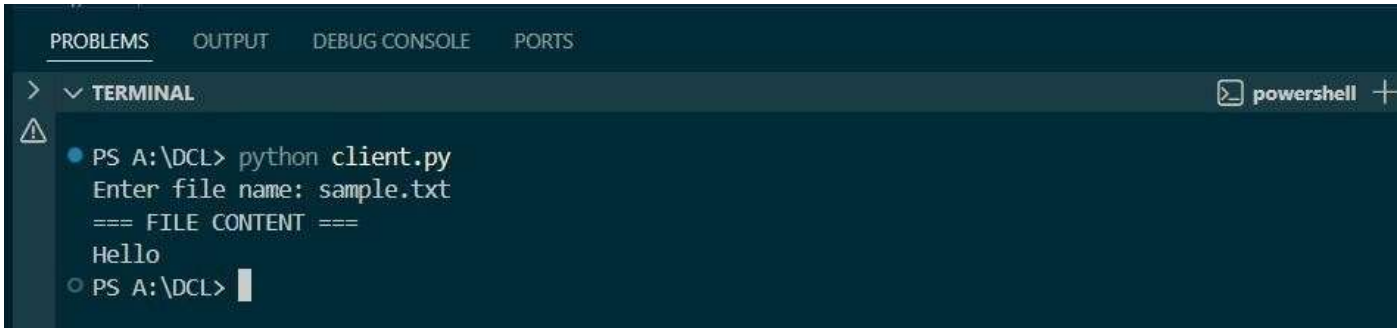


Client code:

```
import socket

HOST = '127.0.0.1'
PORT = 5000
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

file_name = input("Enter file name: ")
client.send(file_name.encode())

response = client.recv(4096).decode()

if response.startswith("FILE_FOUND"):
    print("=== FILE CONTENT ===")
    print(response.split("\n", 1)[1])
else:
    print("Error:", response)
client.close()
```

OUTPUT

Command: python client.py



4. CONCLUSION

The Inter Process Communication using the pipe mechanism was successfully implemented. The pipe was created to establish communication between the parent and child processes. The parent process was able to send data through the write end of the pipe. The child process successfully received the data from the read end of the pipe. This experiment demonstrated unidirectional communication between processes. The FIFO nature of the pipe ensured correct order of data transfer. Proper closing of unused pipe ends prevented deadlock situations. Thus, the pipe mechanism proved to be an effective IPC method for local process communication.

Sign and Remarks:

R1 (4 Marks)	R2 (4 Marks)	R3 (4 Marks)	R4 (3 Marks)	Total Marks (15 Marks)	Signature